Universidad de Ingeniería y Tecnología (UTEC)
Facultad de Ciencia de la Computación

CS4052 – Computación Paralela y Distribuida – 2026-II

Informe final de proyecto

Proyecto 5: paralelización del clasificador KNN sobre `load_digits` mediante MPI (`mpi4py`)

Integrantes:

Juan David Velo Poma	(100 %)
Farid Jack Aquino Castro	(100 %)
Josue Nehemias Velo Poma	(100 %)

Profesor:
José Fiestas, PhD

Entrega:
Noviembre de 2026

Índice

1. Introducción **2**
1.1. El problema y el algoritmo elegido . . . 2
1.2. Código secuencial de partida . . . 2
1.3. Solución planteada . . . 2

2. Método **3**
2.1. Notación . . . 3
2.2. Metodología de Foster (PCAM) y modelo de ejecución . . . 3
2.3. Pseudocódigos: secuencial, PRAM y distribuido con `pardo` . . . 3
2.4. Análisis de complejidad (resumen del informe parcial) . . . 6
2.5. Código: desarrollo en tres versiones beta . . . 7
2.6. Datos sintéticos a escala . . . 8
2.7. Entorno experimental y protocolo . . . 8

3. Resultados y análisis **9**
3.1. Registro de versiones . . . 9
3.2. Tiempos de ejecución, cómputo y comunicación . . . 10
3.3. Normalización del modelo teórico y cantidad óptima de procesos . . . 10
3.4. Speedup y Ley de Amdahl (escalabilidad fuerte) . . . 12
3.5. Eficiencia . . . 13
3.6. Rendimiento en FLOP/s . . . 14
3.7. Escalabilidad débil y Ley de Gustafson . . . 14
3.8. Modelo de comunicación: árbol binario frente a lineal . . . 15
3.9. Justificación de hardware: núcleos P, núcleos E y SMT . . . 16
3.10. Comparación de descomposiciones: Beta 3 frente a variante DAG . . . 17
3.11. Límites de escalabilidad y mejoras propuestas . . . 18

4. Conclusiones **18**

5. Bibliografía y uso de fuentes externas **18**
5.1. Material del curso . . . 18
5.2. Literatura científica y documentación técnica . . . 19
5.3. Uso de herramientas de asistencia y modelos de lenguaje . . . 19

Referencias **20**

1. Introducción

1.1. El problema y el algoritmo elegido

El algoritmo de k vecinos más cercanos (KNN), propuesto por Fix y Hodges [1] y caracterizado formalmente por Cover y Hart [2], es un clasificador supervisado no paramétrico: no construye un modelo durante el entrenamiento y difiere todo el cómputo al momento de la consulta. Para clasificar $q \in \mathbb{R}^d$ se calcula su distancia a los n_{train} puntos etiquetados, se eligen los k más cercanos y se asigna la etiqueta mayoritaria.

Este perfil —entrenamiento gratuito, inferencia costosa— lo hace idóneo para el cómputo paralelo: el trabajo se concentra en una región bien identificada (las $m_{\text{test}} \cdot n_{\text{train}}$ distancias) y cada consulta es independiente de las demás. En el modelo DAG del curso, el problema es un algoritmo secuencial–paralelo: distribución de datos, una fase masivamente paralela y recolección de resultados [3].

1.2. Código secuencial de partida

El archivo `knn_digits_sec.py` del enunciado [4] usa `load_digits` [5, 6]: 1797 imágenes de 8×8 píxeles con valores en $[0, 16]$ ($d = 64$, $C = 10$ clases). Con `test_size=0.2` y `random_state=42` se obtiene $n_{\text{train}} = 1437$ y $m_{\text{test}} = 360$, y se fija $k = 3$. La función `knn_predict` recorre en un bucle de Python los n_{train} puntos calculando

$$d(q, x) = \sqrt{\sum_{l=1}^{d} (q_l - x_l)^2}, \tag{1}$$

ordena las distancias con `np.argsort` y vota con `Counter`. Su exactitud es 0.9833 y su tiempo, $T_s = 1.40\,\text{s}$ en la plataforma de la Sec. 2.7.

1.3. Solución planteada

Se descompone el dominio de consultas: X_{te} se reparte en bloques disjuntos entre los s procesos MPI y X_{tr}, que todo proceso necesita íntegro, se replica con una difusión (`bcast`/`Bcast`). Las predicciones locales se recolectan con `gather`/`Gatherv`. Además, cumpliendo lo anunciado en el informe parcial, se implementa la descomposición alternativa del DAG del enunciado —reparto de X_{tr}, difusión de X_{te} y fusión de candidatos— y se comparan ambas experimentalmente.

El desarrollo se registra en tres versiones beta (Sec. 2.5). Como `load_digits` se resuelve en milisegundos y el arranque de MPI dominaría las mediciones, se generan conjuntos sintéticos de hasta $N = 245\,000$ muestras con la misma estadística por clase (Sec. 2.6).

2. Método

2.1. Notación

Tabla 1: Notación empleada en todo el informe (idéntica al informe parcial).

Símbolo	Significado
n_{train} (o n)	puntos de entrenamiento
m_{test} (o m)	puntos de prueba (consultas)
$N = n_{\text{train}} + m_{\text{test}}$	tamaño total del conjunto ($n_{\text{train}} = 0.8N$)
d	dimensión del espacio de características ($d = 64$)
k	número de vecinos ($k = 3$)
s	número de procesos MPI (el p del enunciado)
p	rango de un proceso, $p \in \{0, \ldots, s-1\}$
m_p, displ_p	consultas asignadas al proceso p y su desplazamiento
α, β	latencia por mensaje y tiempo por palabra
W, T_∞	trabajo y span (camino crítico)
T_s, $T_p(s)$	tiempo secuencial y paralelo con s procesos
T_{comp}, T_{comm}	tiempos de cómputo y de comunicación
$S(s)$, $E(s)$	speedup y eficiencia
f_s, f_p	fracciones serial y paralela ($f_s + f_p = 1$)
k_1, k_2	constantes de normalización del modelo teórico

2.2. Metodología de Foster (PCAM) y modelo de ejecución

El diseño sigue la metodología de Foster [7, 3]:

1. **Particionamiento.** La tarea primitiva es el cálculo de la distancia (Ec. 1) entre una consulta y un punto de entrenamiento: $m_{\text{test}} \cdot n_{\text{train}}$ tareas.

2. **Comunicación.** Todas las tareas de una consulta leen X_{tr} completo (comunicación global $\Rightarrow$ `Bcast`); cada consulta se envía a un solo proceso (`Scatterv`); cada etiqueta predicha vuelve a la raíz (`Gatherv`).

3. **Aglomeración.** Las n_{train} distancias de una consulta y las $m_p \approx m_{\text{test}}/s$ consultas de un bloque se agrupan en un proceso, maximizando la granularidad $G = T_{\text{comp}}/T_{\text{comm}}$.

4. **Mapeo.** Un proceso MPI por núcleo lógico del procesador multinúcleo, con reparto por bloques que admite m_{test} mód $s \neq 0$.

En la taxonomía de Flynn [8] el sistema es MIMD con paradigma SPMD: los s procesos ejecutan el mismo programa sobre datos distintos.

2.3. Pseudocódigos: secuencial, PRAM y distribuido con `pardo`

Los Algoritmos 1–3 describen el código secuencial. Atendiendo la retroalimentación de la entrega parcial, **todos los bucles concurrentes se escriben con el formalismo `forall ... pardo`** [9, 10]: en la PRAM (Algoritmo 4) y en las dos versiones distribuidas (Algoritmos 5 y 6).

Algorithm 1: Distancia euclidiana

Input: Dos arreglos $a, b \in \mathbb{R}^d$
Output: Distancia $\|a - b\|_2$

$\delta \leftarrow 0$;
for $i \leftarrow 0$ **to** $d - 1$ **do**
  $\delta \leftarrow \delta + (a[i] - b[i])^2$;
return $\sqrt{\delta}$;

Algorithm 2: KNN predict

Input: Punto de prueba $x_{\text{test}} \in \mathbb{R}^d$, $X_{\text{tr}} \in \mathbb{R}^{n_{\text{train}} \times d}$, $y_{\text{tr}} \in \{0, \ldots, 9\}^{n_{\text{train}}}$, número de vecinos k
Output: Etiqueta predicha $\hat{y}$ (moda de los k vecinos más cercanos)

$D \leftarrow [\,]$;
foreach $x_{punto} \in X_{tr}$ **do**
  añadir `Distancia(`x_{test}, x_{punto}`)` a D ; ▷ `Algoritmo 1`
$k_indices \leftarrow$ `argsort`$(D)[0..k-1]$;
$k_labels \leftarrow [\, y_{\text{tr}}[i] : i \in k_indices \,]$;
return *moda*(k_labels);

Algorithm 3: KNN secuencial

Input: Datos $X \in \mathbb{R}^{(n_{\text{train}}+m_{\text{test}}) \times d}$ y etiquetas $y \in \{0, \ldots, 9\}^{n_{\text{train}}+m_{\text{test}}}$
Output: Exactitud del modelo y tiempo de ejecución

$X_{\text{tr}}, X_{\text{te}}, y_{\text{tr}}, y_{\text{te}} \leftarrow$ `train_test_split`$(X, y,$ `test_size` $= 0.2,$ `random_state` $= 42)$;
$k \leftarrow 3$; $t_{\text{inicio}} \leftarrow$ `time`$()$;
$\hat{y} \leftarrow [\,$`knn_predict(`x, X_{tr}, y_{tr}, k`)` $: x \in X_{\text{te}}\,]$; ▷ `Algoritmo 2`
exactitud $\leftarrow$ `mean`$(\hat{y} = y_{\text{te}})$; $t_{\text{fin}} \leftarrow$ `time`$()$;
return *exactitud,* $t_{fin} - t_{inicio}$;

Algorithm 4: KNN en PRAM CREW

Input: Datos $X \in \mathbb{R}^{(n_{\text{train}}+m_{\text{test}}) \times d}$ y etiquetas $y \in \{0, \ldots, 9\}^{n_{\text{train}}+m_{\text{test}}}$
Output: Exactitud del modelo y tiempo de ejecución
Memoria global: $X_{\text{tr}}, y_{\text{tr}}$ (CR); $\hat{y}$ (EW)

$X_{\text{tr}}, X_{\text{te}}, y_{\text{tr}}, y_{\text{te}} \leftarrow$ `train_test_split`$(X, y,$ `test_size` $= 0.2,$ `random_state` $= 42)$;
$k \leftarrow 3$;
$t_{\text{inicio}} \leftarrow$ `time`$()$;
$\hat{y} \leftarrow 0 \in \{0, \ldots, 9\}^{m_{\text{test}}}$;
forall $j \leftarrow 1$ **to** m_{test} **pardo**
  $\hat{y}[j] \leftarrow$ `knn_predict(`$X_{te}[j], X_{tr}, y_{tr}, k$`)` ; ▷ `procesador` j`; Algoritmo 2`
exactitud $\leftarrow$ `mean`$(\hat{y} = y_{\text{te}})$;
$t_{\text{fin}} \leftarrow$ `time`$()$;
return *exactitud,* $t_{fin} - t_{inicio}$;

Algorithm 5: KNN distribuido con MPI: replicación de X_{tr}, partición de X_{te}

Input: Datos X, etiquetas y, s procesos, $k = 3$
Output: Exactitud del modelo y tiempo de ejecución (en el proceso 0)
Datos locales: El proceso p recibe $m_p = \lfloor m_{\text{test}}/s \rfloor + [\, p < m_{\text{test}} \bmod s \,]$ consultas desde $\text{displ}_p = \sum_{i<p} m_i$

forall $p \leftarrow 0$ **to** $s-1$ **pardo**
  if $p = 0$ **then**
    $X_{\text{tr}}, X_{\text{te}}, y_{\text{tr}}, y_{\text{te}} \leftarrow$ `train_test_split`$(X, y,$ `test_size` $= 0.2,$ `random_state` $=$ $42)$;
    $t_{\text{inicio}} \leftarrow$ `Wtime`();
  ; ▷ `replicación:` $\log s\,(\alpha + n_{\texttt{train}} d\,\beta)$
  $X_{\text{tr}}, y_{\text{tr}} \leftarrow$ `Bcast`$(X_{tr},\, y_{tr},\, raíz = 0)$;
  ; ▷ `partición de consultas:` $\alpha \log s + m_{\texttt{test}} d\,\beta$
  $X_{\text{te}}^{(p)} \leftarrow$ `Scatterv`$(X_{te},\, (m_i d)_i,\, (\text{displ}_i d)_i,\, raíz = 0)$;
  ; ▷ `cómputo local:` $m_p\, n_{\texttt{train}}\,(3d)$ `FLOPs`
  forall $j \leftarrow 1$ **to** m_p **pardo**
    $\hat{y}^{(p)}[j] \leftarrow$ `knn_predict`$(X_{te}^{(p)}[j], X_{tr}, y_{tr}, k)$; ▷ `Algoritmo 2; CR`
  ; ▷ `recolección:` $\alpha \log s + m_{\texttt{test}}\,\beta$
  $\hat{y}[\text{displ}_p + 1\,..\,\text{displ}_p + m_p] \leftarrow$ `Gatherv`$(\hat{y}^{(p)},\, (m_i)_i,\, (\text{displ}_i)_i,\, raíz = 0)$; ▷ `EW: bloques disjuntos`
  if $p = 0$ **then**
    exactitud $\leftarrow$ `mean`$(\hat{y} = y_{\text{te}})$;
    $t_{\text{fin}} \leftarrow$ `Wtime`();
    return *exactitud,* $t_{fin} - t_{inicio}$;

Formalismo `pardo`. En el Algoritmo 4 el índice j recorre los m_{test} procesadores de una PRAM CREW [9]: todos leen concurrentemente X_{tr} y y_{tr} (CR) y el procesador j escribe sólo en $\hat{y}[j]$ (EW). En los Algoritmos 5 y 6, el bucle externo **forall** $p \leftarrow 0$ **to** $s-1$ **pardo** expresa el paradigma SPMD: los s procesos ejecutan concurrentemente el mismo cuerpo y se distinguen sólo por su rango; las colectivas son puntos de sincronización implícita. El bucle interno expresa que las consultas locales son independientes (no hay aristas entre ellas en el DAG). Como cada proceso ocupa un único núcleo, ese `pardo` interno se ejecuta como paralelismo de datos —producto matricial por bloques— y, por el teorema de Brent [11],

$$T_{\text{comp}}(s) \leq \frac{W}{s} + T_{\infty}. \tag{2}$$

Con m_p procesadores virtuales por proceso ($s\, m_p = m_{\text{test}}$), el Algoritmo 5 se reduce exactamente al Algoritmo 4.

Algorithm 6: KNN distribuido con MPI según el DAG: partición de X_{tr}, difusión de X_{te}

Input: Datos X, etiquetas y, s procesos, $k = 3$
Output: Exactitud del modelo y tiempo de ejecución (en el proceso 0)
Datos locales: El proceso p recibe $n_p = \lfloor n_{\text{train}}/s \rfloor + [\, p < n_{\text{train}} \bmod s \,]$ puntos de entrenamiento $(X_{\text{tr}}^{(p)}, y_{\text{tr}}^{(p)})$

forall $p \leftarrow 0$ **to** $s-1$ **pardo**
  if $p = 0$ **then**
    $X_{\text{tr}}, X_{\text{te}}, y_{\text{tr}}, y_{\text{te}} \leftarrow$ `train_test_split`$(X, y,$ `test_size` $= 0.2,$ `random_state` $=$ $42)$;
    $t_{\text{inicio}} \leftarrow$ `Wtime`();
  ; ▷ `partición del entrenamiento:` $\alpha \log s + n_{\texttt{train}} d\, \beta$
  $X_{\text{tr}}^{(p)}, y_{\text{tr}}^{(p)} \leftarrow$ `Scatterv`$(X_{tr}, y_{tr},$ *raíz* $= 0)$;
  ; ▷ `difusión de consultas:` $\log s\, (\alpha + m_{\texttt{test}} d\, \beta)$
  $X_{\text{te}} \leftarrow$ `Bcast`$(X_{te},$ *raíz* $= 0)$;
  ; ▷ `cómputo local:` $m_{\texttt{test}}\, n_p\, (3d)$ `FLOPs`
  forall $j \leftarrow 1$ **to** m_{test} **pardo**
    $(\delta_j^{(p)}, \ell_j^{(p)}) \leftarrow k$ menores distancias de $X_{\text{te}}[j]$ a $X_{\text{tr}}^{(p)}$ y sus etiquetas;
  ; ▷ `recolección de candidatos:` $\alpha \log s + m_{\texttt{test}}\, k\, s\, \beta$
  $\{(\delta^{(i)}, \ell^{(i)})\}_{i=0}^{s-1} \leftarrow$ `Gather`$((\delta^{(p)}, \ell^{(p)}),$ *raíz* $= 0)$;
  if $p = 0$ **then**
    for $j \leftarrow 1$ **to** m_{test} **do**
      $\hat{y}[j] \leftarrow$ `moda`$\big(k$ etiquetas de menor distancia entre los $s\,k$ candidatos de $j\big)$; ▷ `fusión serial` $O(s\,k)$
    exactitud $\leftarrow$ `mean`$(\hat{y} = y_{\text{te}})$;
    $t_{\text{fin}} \leftarrow$ `Wtime`();
    return *exactitud,* $t_{fin} - t_{inicio}$;

Corrección de la variante DAG. Si x pertenece a los k vecinos globales de q y $x \in X_{\text{tr}}^{(p)}$, entonces x también está entre los k vecinos de q dentro del bloque p. Luego la unión de los $s\,k$ candidatos contiene a los k vecinos globales y la predicción coincide con la secuencial (salvo empates exactos de distancia).

2.4. Análisis de complejidad (resumen del informe parcial)

Del informe parcial se conservan el trabajo $W = T_s = O\big(m_{\text{test}}(n_{\text{train}} d + n_{\text{train}} \log n_{\text{train}})\big)$, el span $T_\infty = O(n_{\text{train}} d + n_{\text{train}} \log n_{\text{train}})$ y el tiempo paralelo del Algoritmo 5 con colectivas en árbol [12]:

$$T_p(s) = O\Big(\frac{m_{\text{test}}}{s}\, n_{\text{train}} d + \alpha \log s + \beta\, (n_{\text{train}} d \log s + m_{\text{test}} d)\Big). \tag{3}$$

En la implementación final, la selección de los k vecinos usa `np.argpartition`, $O(n_{\text{train}})$, por lo que el término $n_{\text{train}} \log n_{\text{train}}$ desaparece. Para la variante DAG (Algoritmo 6):

$$T_{\text{comm}}^{\text{DAG}}(s) = O\big(\alpha \log s + \beta\, (n_{\text{train}} d + m_{\text{test}} d \log s + m_{\text{test}} k s)\big), \qquad T_{\text{fusión}} = O(m_{\text{test}}\, s\, k), \tag{4}$$

es decir, evita difundir X_{tr} ($n_{\text{train}} = 4 m_{\text{test}}$) a cambio de una recolección y una fusión serial que crecen linealmente con s.

FLOPs de la región paralelizable. Por cada par (consulta, punto de entrenamiento), la Ec. (1) realiza d restas, d multiplicaciones, $d-1$ sumas y una raíz cuadrada (IEEE 754): $3d$

FLOPs. El total y el rendimiento son

$$W_{\text{FLOP}} = 3\, m_{\text{test}}\, n_{\text{train}}\, d, \qquad \text{GFLOP/s} = \frac{3\, m_{\text{test}}\, n_{\text{train}}\, d}{T_{\text{comp}} \cdot 10^9}. \tag{5}$$

La implementación vectorizada calcula $\|q - x\|^2 = \|q\|^2 + \|x\|^2 - 2\, q \cdot x$ mediante productos matriciales; por ello se reportan *FLOPs útiles* según la Ec. (5), que es la métrica pedida por el enunciado e independiente de la implementación.

2.5. Código: desarrollo en tres versiones beta

Todo el código está en `src/` y se resume en la Tabla 2.

Tabla 2: Registro del desarrollo del código.

Versión	Cambio principal	Motivación
Secuencial	Código original del enunciado (bucle Python por punto).	Línea base T_s y exactitud de referencia.
Beta 1 (`knn_mpi_v1.py`)	`comm.bcast`, `comm.scatter` y `comm.gather` (objetos *pickle*); `knn_predict` idéntica a la secuencial.	Aislar el efecto de la descomposición de datos.
Beta 2 (`knn_mpi_v2.py`)	Distancias por bloques con productos matriciales (NumPy), `argpartition`, voto vectorizado; medición separada de T_{comp} y T_{comm} con `MPI.Wtime()`.	El bucle Python es ∼150 veces más lento que el cómputo vectorizado.
Beta 3 (`knn_mpi_v3_optimized.py`)	`Bcast`, `Scatterv` y `Gatherv` con buffers contiguos; counts/displacements para m_{test} mód $s \neq 0$; FLOPs y CSV.	Eliminar la serialización; el costo sigue el modelo α–β.
DAG (`knn_mpi_dag.py`)	`Scatterv`(X_{tr}), `Bcast`(X_{te}), `Gather` de k candidatos y fusión en la raíz.	Descomposición del DAG del enunciado.

El núcleo de la Beta 3 (una ejecución del Algoritmo 5) es:

```
rows, displs = block_partition(m, size) # m_p = q+1 si p < r, q en otro caso
comm.Barrier(); t0 = MPI.Wtime()
comm.Bcast([X_train, MPI.DOUBLE], root=0) # (1) replicación de X_tr
comm.Bcast([y_train, MPI.INT64_T], root=0)
t1 = MPI.Wtime()
sendbuf = [X_test, (rows*d, displs*d), MPI.DOUBLE] if rank == 0 else None
comm.Scatterv(sendbuf, [X_local, MPI.DOUBLE], root=0) # (2) bloque de consultas
t2 = MPI.Wtime()
y_local = knn_local(X_train, y_train, X_local, k, block) # (3) forall j pardo
t3 = MPI.Wtime()
recvbuf = [y_pred, (rows, displs), MPI.INT64_T] if rank == 0 else None
comm.Gatherv([y_local, MPI.INT64_T], recvbuf, root=0) # (4) EW en bloques disjuntos
t4 = MPI.Wtime()
```

y el cómputo local de un bloque de consultas Q:

```
D2 = np.einsum("ij,ij->i", Q, Q)[:, None] + sq_train[None, :]
D2 -= 2.0 * (Q @ X_train.T) # producto matricial (BLAS-3)
D = np.sqrt(np.maximum(D2, 0.0)) # Ec. (1)
nn = np.argpartition(D, k - 1, axis=1)[:, :k] # k vecinos en O(n)
```

```
votes = np.zeros((Q.shape[0], 10), dtype=np.int64)
np.add.at(votes, (np.arange(Q.shape[0])[:, None], y_train[nn]), 1)
pred = votes.argmax(axis=1) # moda
```

El reparto por bloques garantiza que el proceso p escribe sólo en $\hat{y}[\text{displ}_p + 1..\text{displ}_p + m_p]$: es la escritura exclusiva de la PRAM CREW.

Validación. Las cinco versiones obtienen exactitud 0.9833 en `load_digits`, igual a la secuencial. Las Betas 2–3 y la variante DAG se validaron además contra `KNeighborsClassifier` de scikit-learn [5] (`algorithm="brute"`, métrica euclidiana): coincidencia del 100 % de las predicciones en `load_digits` ($s = 1, 7$) y en $N = 50\,000$ ($s = 1, 8$), incluido un reparto no divisible (360 consultas entre 7 procesos).

2.6. Datos sintéticos a escala

`data_generator.py` estima, para cada clase c de `load_digits`, la probabilidad a priori π_c, la media μ_c y la covarianza Σ_c, y genera $y \sim \text{Cat}(\pi)$, $x \mid y = c \sim \mathcal{N}(\mu_c, \Sigma_c + \varepsilon I)$, recortando y redondeando a $[0, 16]$ ($\varepsilon = 10^{-3}$ regulariza Σ_c, singular por píxeles de borde constantes). Se conservan $d = 64$ y $C = 10$. Al ser gaussianas por clase, estos datos son más separables que los reales (exactitud 0.998–0.999); por eso la exactitud del modelo se reporta sobre `load_digits` y los datos sintéticos se usan sólo para medir rendimiento.

2.7. Entorno experimental y protocolo

- **Hardware:** laptop ASUS TUF Gaming F16, Intel Core i7-14650HX [13]: arquitectura híbrida con 8 P-cores (2 hilos SMT cada uno) y 8 E-cores (1 hilo), 24 hilos lógicos; 32 GB DDR5-5600. La topología se obtuvo con `GetSystemCpuSetInformation` [14]: CPUs lógicas 0–15 = P-cores (pares SMT $\{0, 1\}, \{2, 3\}, \dots$) y 16–23 = E-cores.
- **Software:** Windows 11, Python 3.11.9, NumPy [15], mpi4py 4.1.2 [16, 17] sobre Intel MPI 2021.19 [18]. Las variables `OMP_NUM_THREADS`, `MKL_NUM_THREADS` y `OPENBLAS_NUM_THREADS` se fijan en 1, de modo que cada proceso MPI usa un único núcleo.
- **Barrido:** $s \in \{1, 2, 4, 8, 12, 16, 20, 24\}$. Escalabilidad fuerte con $N = 50\,000$, $100\,000$ y $200\,000$; débil con $N = N_0\sqrt{s}$ ($N_0 = 50\,000$) y con $N = N_0\, s$ ($N_0 = 10\,000$).
- **Medición:** `Barrier` y `MPI.Wtime()` por fase; se reporta el máximo entre procesos y la mediana de 3 repeticiones (7 para $N = 50\,000$). T_{comm} incluye la espera en las colectivas por el proceso más lento.
- **Experimentos complementarios:** (i) colectivas sin cómputo (`comm_bench.py`) para medir α y β; (ii) carga fija por proceso con afinidad explícita a P-cores, E-cores o hilos SMT (`pin_bench.py`).
- **Automatización:** `benchmark_runner.py` y `extra_benchmarks.py` (reanudables, CSV consolidado); `generate_plots.py` produce figuras, ajustes y tablas con SciPy [19] y Matplotlib [20].

La campaña completa (∼60 min) comprende 291 ejecuciones de escalabilidad y versiones, más 16 configuraciones de afinidad y 7 barridos de comunicación.

3. Resultados y análisis

3.1. Registro de versiones

Tabla 3: Evolución de las versiones sobre `load_digits` ($n_{\text{train}} = 1437$, $m_{\text{test}} = 360$). Speedup respecto del secuencial original ($T_s = 1.401\,\text{s}$).

Versión	s	T_{total} [s]	Speedup	Exactitud	Validación
Secuencial (original)	1	1.401	1.0	0.9833	referencia
Beta 1	1	1.4022	1.0	0.9833	= secuencial
Beta 1	8	0.2099	6.7	0.9833	= secuencial
Beta 2	1	0.0090	155.7	0.9833	= secuencial
Beta 2	8	0.0030	467.1	0.9833	= secuencial
Beta 3	1	0.0078	178.8	0.9833	100 % = sklearn
Beta 3	8	0.0020	699.8	0.9833	100 % = sklearn
DAG	1	0.0083	169.6	0.9833	100 % = sklearn
DAG	8	0.0014	1014.1	0.9833	100 % = sklearn

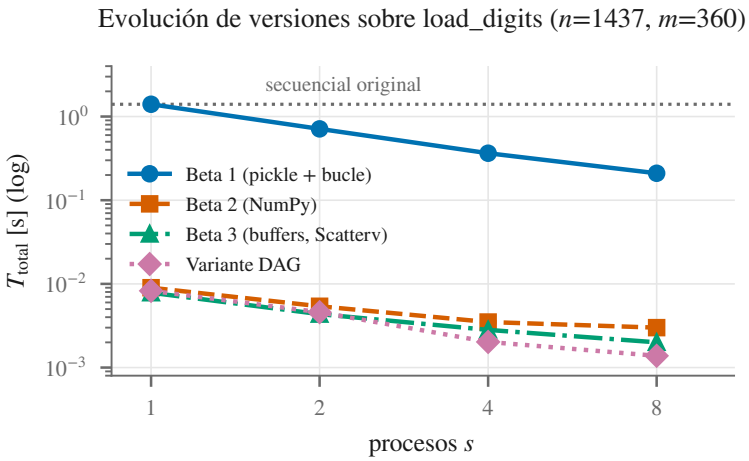


Figura 1: Tiempo total por versión y número de procesos sobre `load_digits`.

En la Beta 1 el cómputo por consulta es muy costoso, la granularidad $G = T_{\text{comp}}/T_{\text{comm}}$ es alta y la eficiencia con $s = 8$ llega a 0.83. La Beta 2 reduce el trabajo efectivo en dos órdenes de magnitud al sustituir $m_{\text{test}} n_{\text{train}}$ llamadas Python por productos matriciales; con ello `load_digits` se resuelve en milisegundos, la granularidad cae y la latencia de las colectivas pasa a dominar. Esta es la justificación cuantitativa para escalar N con datos sintéticos. La Beta 3 elimina la serialización *pickle* y reduce el tiempo un 33 % adicional con $s = 8$.

3.2. Tiempos de ejecución, cómputo y comunicación

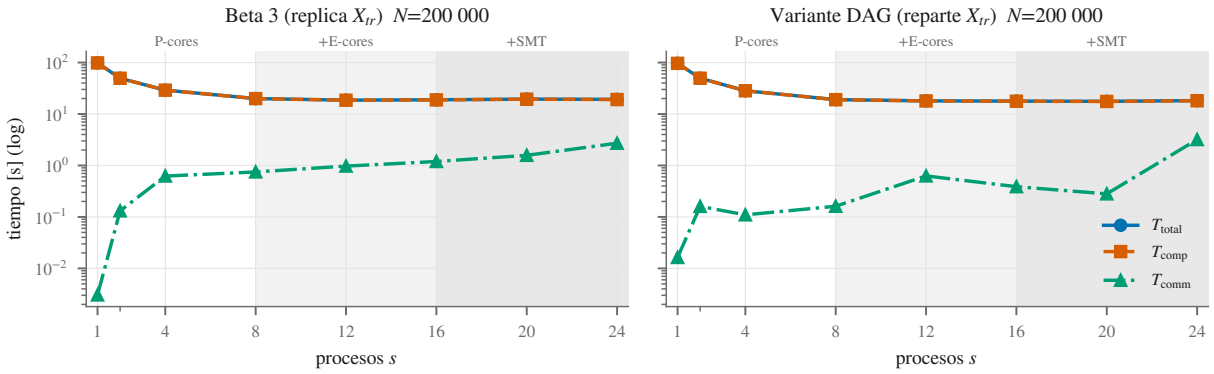


Figura 2: T_{total}, T_{comp} y T_{comm} frente a s con $N = 200\,000$ ($n_{\text{train}} = 160\,000$, $m_{\text{test}} = 40\,000$). Las bandas sombreadas indican las regiones de hardware: hasta 8 procesos caben en P-cores físicos; de 8 a 16 se incorporan E-cores; por encima de 16 se usan hilos SMT.

Tabla 4: Escalabilidad fuerte de la Beta 3 con $N = 200\,000$ (medianas).

s	T_{total} [s]	T_{comp} [s]	T_{comm} [s]	$S(s)$	$E(s)$	GFLOP/s
1	98.46	98.46	0.003	1.00	1.00	12.5
2	49.53	49.51	0.131	1.99	0.99	24.8
4	29.09	29.06	0.623	3.39	0.85	42.3
8	19.97	19.93	0.749	4.93	0.62	61.7
12	18.60	18.54	0.972	5.29	0.44	66.3
16	18.90	18.80	1.195	5.21	0.33	65.4
20	19.50	19.39	1.574	5.05	0.25	63.4
24	19.26	19.11	2.705	5.11	0.21	64.3

El cómputo representa más del 99 % de T_{total} en todo el rango, lo que confirma la granularidad gruesa obtenida en la aglomeración. Sin embargo, T_{comp} deja de disminuir a partir de $s = 8$: cada proceso realiza W/s operaciones pero tarda lo mismo, por lo que el límite no está en el algoritmo sino en la plataforma. El crecimiento de T_{comm} para $s \geq 16$ corresponde principalmente a espera en `Gatherv` por desbalance de carga (Sec. 3.9).

3.3. Normalización del modelo teórico y cantidad óptima de procesos

Modelo. A partir de la Ec. (3), el tiempo paralelo se expresa como la suma de un término de cómputo y uno de comunicación, cada uno multiplicado por una constante de normalización que convierte operaciones y palabras en segundos de esta máquina:

$$T_p(s) = k_1 \frac{W}{s} + k_2\, T_{\text{comm}}(s), \qquad W = 3\, m_{\text{test}}\, n_{\text{train}}\, d. \tag{6}$$

Para $T_{\text{comm}}(s)$ se comparan los dos modelos de comunicación vistos en clase, "Comunicación lineal" y "Comunicación en árbol (binario)" [21], aplicados a la difusión, que es la colectiva dominante:

$$\text{árbol binario:} \quad T_{\text{comm}}(s) = \log_2 s\,(\alpha + M\beta), \tag{7}$$

$$\text{lineal:} \quad T_{\text{comm}}(s) = (s-1)\,(\alpha + M\beta), \tag{8}$$

con $M = n_{\text{train}}(d+1) + m_{\text{test}}d + m_{\text{test}}$ palabras (`Bcast` de $X_{\text{tr}}, y_{\text{tr}}$, `Scatterv` de X_{te} y `Gatherv` de $\hat{y}$).

Procedimiento de normalización.

1. α y β se miden independientemente con colectivas sin cómputo (Sec. 3.8), sin usar los tiempos del KNN.
2. Para cada N, se calculan los regresores teóricos $x_1(s) = W/s$ y $x_2(s) = T_{\text{comm}}(s)$.
3. k_1 [s/FLOP] y k_2 [adimensional] se obtienen por mínimos cuadrados no negativos sobre el T_{total} medido, minimizando el error relativo $\sum_s \left[(k_1 x_1 + k_2 x_2 - T_{\text{med}})/T_{\text{med}}\right]^2$, pues los tiempos abarcan dos órdenes de magnitud.
4. $1/k_1$ se interpreta como el rendimiento efectivo por proceso y k_2 como el factor por el que el costo real de compartir datos supera al de las colectivas aisladas.

Proceso óptimo analítico. Derivando la Ec. (6) e igualando a cero, con $c = \alpha + M\beta$:

$$\text{árbol:} \quad -\frac{k_1 W}{s^2} + \frac{k_2\, c}{s \ln 2} = 0 \implies s^* = \frac{k_1\, W \ln 2}{k_2\, c}, \tag{9}$$

$$\text{lineal:} \quad -\frac{k_1 W}{s^2} + k_2\, c = 0 \implies s^* = \sqrt{\frac{k_1\, W}{k_2\, c}}. \tag{10}$$

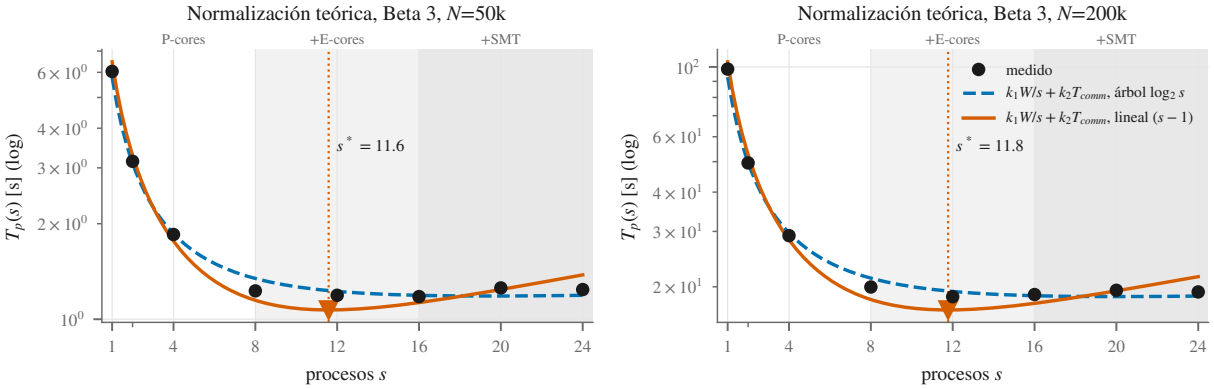


Figura 3: Curvas teóricas normalizadas (Ec. 6) frente a los datos experimentales de la Beta 3. El triángulo marca s^* del modelo lineal (Ec. 10).

Tabla 5: Constantes de normalización y proceso óptimo (Beta 3).

Modelo de T_{comm}	N	k_1 [s/FLOP]	k_2	Error rel. medio	s^*
Árbol, Ec. (9)	50k	7.54×10^{-11}	35.6	3.6 %	19.4
	100k	7.41×10^{-11}	71.8	4.0 %	18.9
	200k	7.56×10^{-11}	139.0	3.4 %	19.9
Lineal, Ec. (10)	50k	8.43×10^{-11}	31.5	6.2 %	**11.6**
	100k	8.30×10^{-11}	63.9	5.7 %	**11.4**
	200k	8.45×10^{-11}	122.0	6.4 %	**11.8**
Mínimo experimental $\arg\min_s T_{\text{total}}$ ($N = 100$k y 200k)					**12**

Discusión.

- $k_1 \approx 8 \times 10^{-11}$ s/FLOP equivale a $\sim$12 GFLOP/s por proceso, el rendimiento medido de un P-core aislado (Sec. 3.9); es estable entre tamaños.

- $k_2 \gg 1$ y crece proporcionalmente a N. Si sólo se pagaran las colectivas, se esperaría $k_2 \approx 1$. El exceso indica que el costo dominante de compartir datos no está en los mensajes MPI, sino en la **competencia de todos los procesos por el mismo bus de memoria** durante el cómputo: cada proceso recorre X_{tr} completo (80 MB con $N = 200$k, más que los 30 MB de caché L3), de modo que el tráfico de memoria es proporcional a N. Es el cuello de botella de von Neumann [22, 3]: la velocidad está limitada por el canal procesador–memoria y no por las unidades aritméticas.
- Con el modelo lineal, como $W \propto N^2$ y $k_2\, c \propto N \cdot N$, el cociente de la Ec. (10) es casi independiente de N: $s^* \approx 11.6$ para los tres tamaños, en excelente acuerdo con el mínimo experimental ($s = 12$). El modelo en árbol ajusta igual de bien los puntos medidos pero, por crecer sólo como $\log s$, no reproduce el aumento de T_p más allá del mínimo y sitúa el óptimo en $s^* \approx 19$–20, en la zona de hilos SMT, donde los datos no muestran mejora.
- **Cantidad óptima estimada de procesos:** $s^* \approx 12$ (8 P-cores + 4 E-cores). Entre $s = 12$ y $s = 24$ el tiempo medido varía menos de un 5 %, por lo que en la práctica conviene usar $s = 12$ y no los 24 hilos disponibles: se obtiene el mismo resultado con la mitad de los recursos.

3.4. Speedup y Ley de Amdahl (escalabilidad fuerte)

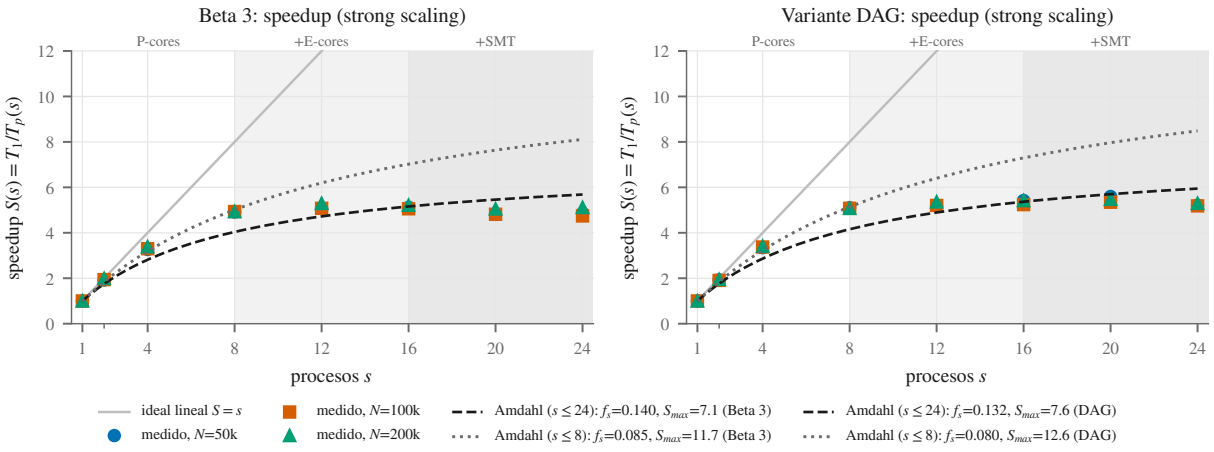


Figura 4: Speedup experimental $S(s) = T_1/T_p(s)$ para tres tamaños, frente al ideal lineal y a la Ley de Amdahl ajustada en todo el rango y sólo en la región de P-cores ($s \le 8$).

La Ley de Amdahl [23, 24] modela el speedup con tamaño fijo:

$$S(s) = \frac{1}{f_s + \dfrac{f_p}{s}}, \qquad f_s + f_p = 1, \qquad S_{\text{máx}} = \lim_{s\to\infty} S(s) = \frac{1}{f_s}. \tag{11}$$

Como T_s se usa la Beta 3 con $s = 1$ (speedup relativo): ejecutar el secuencial original con $N = 200\,000$ tomaría ∼5 h, unas 12 400 veces el caso de `load_digits`. El speedup absoluto respecto del secuencial original se reporta en la Tabla 3.

Tabla 6: Fracción serial ajustada por la Ley de Amdahl ($N = 200\,000$).

Versión	Rango de ajuste	f_s	$S_{\text{máx}} = 1/f_s$	S máximo medido
Beta 3	$1 \le s \le 24$	0.140	7.1	5.29 ($s = 12$)
Beta 3	$1 \le s \le 8$	0.085	11.7	4.93 ($s = 8$)
DAG	$1 \le s \le 24$	0.132	7.6	5.47 ($s = 20$)
DAG	$1 \le s \le 8$	0.080	12.6	5.09 ($s = 8$)

Los tres tamaños de problema producen la misma curva de speedup, que se estanca en 5.1–5.5. La fracción ajustada $f_s \approx 0.14$ **no corresponde a código serial**: la parte secuencial real (`Barrier`, colectivas y cálculo de la exactitud) es menor al 1 % de T_1. Es una *fracción serial efectiva* originada por la contención del bus DDR5: los procesos se turnan para acceder a memoria y ese acceso se comporta como una sección secuencial. Por ello Amdahl no reproduce la forma exacta de los datos ($R^2 = 0.90$) y la fracción crece al incorporar E-cores e hilos SMT ($f_s = 0.085$ en $s \le 8$ frente a 0.140 en todo el rango). Que $S(s)$ no mejore al aumentar N confirma que el límite no es la comunicación MPI, cuyo peso relativo disminuiría con N según la Ec. (3).

3.5. Eficiencia

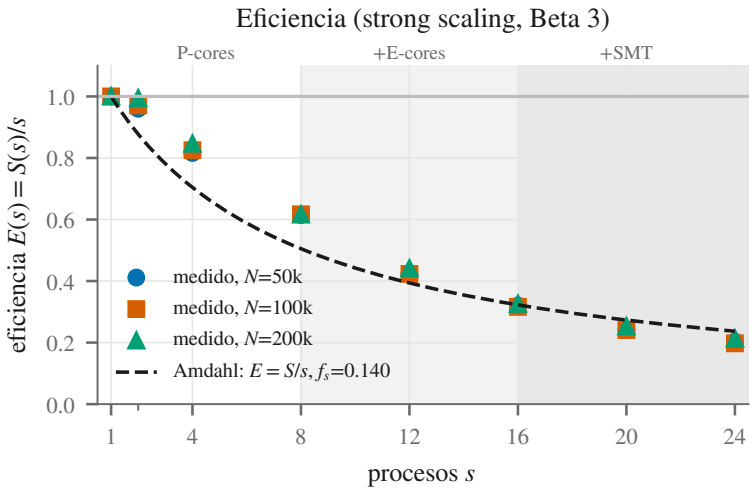


Figura 5: Eficiencia $E(s) = S(s)/s$ de la Beta 3 y predicción de la Ley de Amdahl.

De acuerdo con el análisis del informe parcial, con n_{train} fijo $E(s) \to 0$: el algoritmo *no es fuertemente escalable* en esta plataforma. A partir de $s = 12$ cada proceso adicional aporta menos del 2 % de rendimiento.

3.6. Rendimiento en FLOP/s

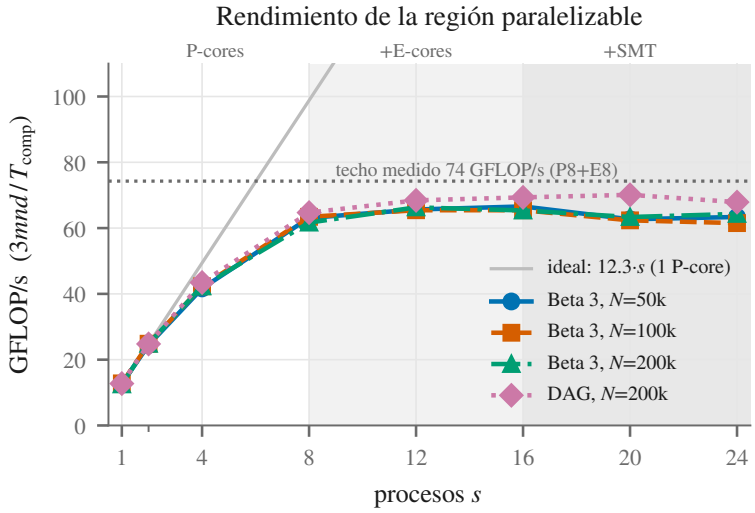


Figura 6: GFLOP/s de la región paralelizable (Ec. 5) frente a s, con el ideal lineal de un P-core y el techo medido con afinidad explícita.

El rendimiento crece casi linealmente hasta $s = 8$ y se estabiliza en 63–66 GFLOP/s (Beta 3) y 68–70 GFLOP/s (DAG), el 90 % del techo medido con carga equilibrada (74.3 GFLOP/s). La curva no depende de N: el rendimiento está determinado por los núcleos activos y su acceso compartido a memoria, no por el tamaño del problema.

3.7. Escalabilidad débil y Ley de Gustafson

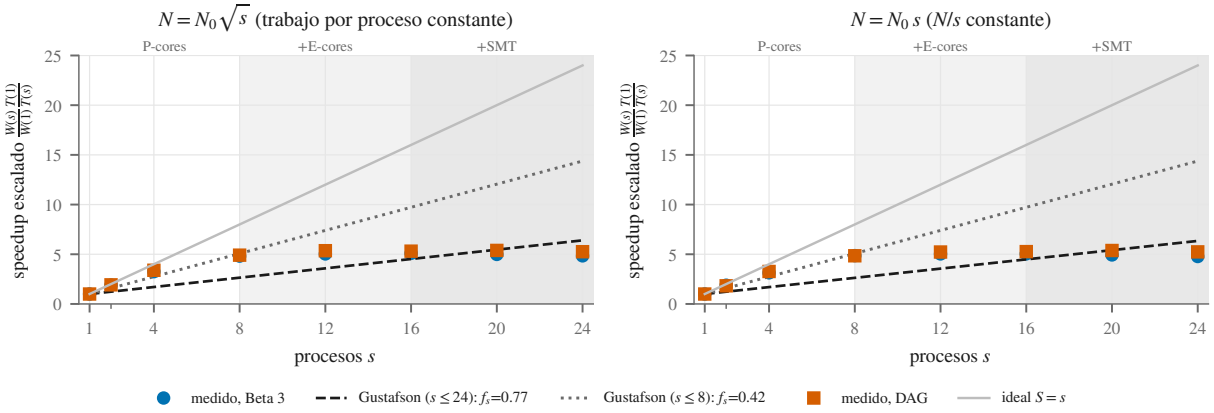


Figura 7: Speedup escalado frente a la Ley de Gustafson. Izquierda: trabajo por proceso constante. Derecha: N/s constante.

La Ley de Gustafson [25, 24] modela el speedup cuando el problema crece con s:

$$S(s) = f_s + s\, f_p = s - f_s\,(s-1). \tag{12}$$

Como el trabajo de KNN es $W = 3m_{\text{test}}n_{\text{train}}d \propto N^2$, mantener N/s constante (como sugiere el enunciado) *no* mantiene constante el trabajo por proceso: $W/s \propto N_0^2\, s$ crece con s. La condición de Gustafson exige $W(s)/s = W(1)$, es decir,

$$N = N_0\sqrt{s}. \tag{13}$$

Se midieron ambas variantes y el speedup escalado se calcula como $S(s) = \frac{W(s)}{W(1)}\frac{T(1)}{T(s)}$, que normaliza el trabajo distinto de cada punto.

Tabla 7: Escalabilidad débil de la Beta 3 con $N = N_0\sqrt{s}$ ($N_0 = 50\,000$).

s	N	T_{total} [s]	Speedup escalado	$E_{\text{débil}}$
1	50 000	5.92	1.0	1.00
2	70 711	6.25	1.9	0.95
4	100 000	7.29	3.2	0.81
8	141 421	9.84	4.8	0.60
12	173 205	14.12	5.0	0.42
16	200 000	18.60	5.1	0.32
20	223 607	23.76	5.0	0.25
24	244 949	29.36	4.8	0.20

Con trabajo por proceso constante el tiempo debería mantenerse; sin embargo, se multiplica por cinco entre $s = 1$ y $s = 24$. El ajuste de la Ec. (12) da $f_s = 0.42$ en $s \leq 8$ y 0.77 en todo el rango. Ambas variantes producen la misma curva normalizada porque miden el mismo fenómeno: la saturación del rendimiento agregado en $\sim$65 GFLOP/s. A diferencia de un clúster, aumentar el problema no añade canales de memoria, de modo que la Ley de Gustafson tampoco se cumple en esta plataforma.

Respecto de la condición de escalabilidad débil del informe parcial (Ec. 18), la comunicación MPI sí permite crecer: con $N = N_0\sqrt{s}$ el `Bcast` de X_{tr} es menor al 1 % de T_{total} incluso con $s = 24$; el resto de T_{comm} es espera por desbalance de carga. El factor limitante es el hardware compartido, no el algoritmo.

3.8. Modelo de comunicación: árbol binario frente a lineal

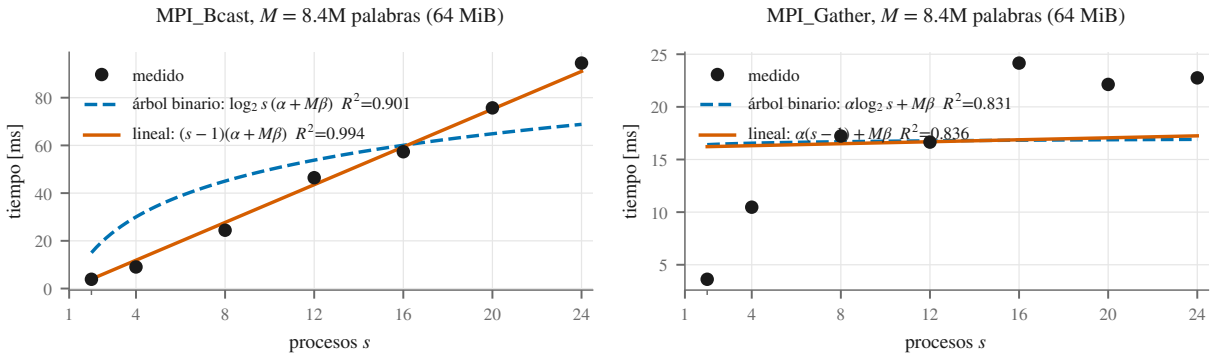


Figura 8: Tiempo de `Bcast` y `Gather` de un mensaje de 64 MiB sin cómputo, frente a los modelos en árbol binario y lineal ajustados con todos los tamaños de mensaje (1 KiB–64 MiB).

Tabla 8: Parámetros de comunicación medidos (ajuste sobre 9 tamaños y 7 valores de s).

Colectiva	Modelo	α [s]	β [s/palabra]	R^2
`Bcast`	árbol, $\log_2 s(\alpha + M\beta)$	≈ 0	1.79×10^{-9}	0.901
	lineal, $(s-1)(\alpha + M\beta)$	≈ 0	4.72×10^{-10}	**0.994**
`Scatter`	árbol, $\alpha \log_2 s + M\beta$	8.2×10^{-5}	1.23×10^{-9}	0.867
	lineal, $\alpha(s-1) + M\beta$	2.9×10^{-5}	1.22×10^{-9}	0.873
`Gather`	árbol, $\alpha \log_2 s + M\beta$	1.4×10^{-4}	1.94×10^{-9}	0.831
	lineal, $\alpha(s-1) + M\beta$	4.7×10^{-5}	1.93×10^{-9}	0.836

En clase se presentaron dos modelos [21]: la *comunicación lineal*, en la que la raíz envía el mensaje a cada proceso por turno, con $T_{\text{bcast}} = O\big(p(\alpha + m\beta)\big)$ y $T_{\text{scatter/gather}} = O(\alpha p + \beta m)$; y la *comunicación en árbol (binario)*, con $T_{\text{bcast}} = O\big(\log p\,(\alpha + m\beta)\big)$ y $T_{\text{scatter/gather}} = O(\alpha \log p + \beta m)$. El modelo en árbol es menor porque en cada etapa se duplica el número de procesos que retransmiten el mensaje por enlaces independientes [12, 26]. En la máquina de este proyecto, el `Bcast` de 64 MiB tarda 3.9 ms con $s = 2$ y 94.5 ms con $s = 24$: crece $\sim$24 veces, lo que corresponde a $O(s)$ y no a $O(\log s)$ ($\log_2 24 / \log_2 2 \approx 4.6$). El modelo lineal explica el 99.4 % de la varianza frente al 90.1 % del árbol.

La razón es que en memoria compartida no existen enlaces independientes: Intel MPI implementa las colectivas intranodo como copias a través de memoria compartida [18], y las $s-1$ copias del mensaje compiten por el mismo canal DDR5. El costo total es entonces $(s-1)M\beta$, con un ancho de banda efectivo por copia de $8/\beta \approx 17$ GB/s. Es otra manifestación del cuello de botella de von Neumann [22]: lo que en un clúster se paralelizaría en la red, aquí se serializa en el bus de memoria. En `Scatter` y `Gather` el término dominante es $M\beta$, común a ambos modelos ($T = O(\alpha p + \beta m)$ frente a $O(\alpha \log p + \beta m)$), por lo que los dos ajustan de forma similar ($R^2 \approx 0.83$–0.87); su variabilidad restante proviene de la sincronización con el proceso más lento.

En el KNN, el `Bcast` de X_{tr} con $N = 200$k (83 MB) cuesta 0.15 s con $s = 24$, sólo el 0.8 % de T_{total}; la comunicación explícita no es el cuello de botella, pero el mismo mecanismo —competencia por el bus de memoria— es el que, durante el cómputo, satura el rendimiento (Sec. 3.3).

3.9. Justificación de hardware: núcleos P, núcleos E y SMT

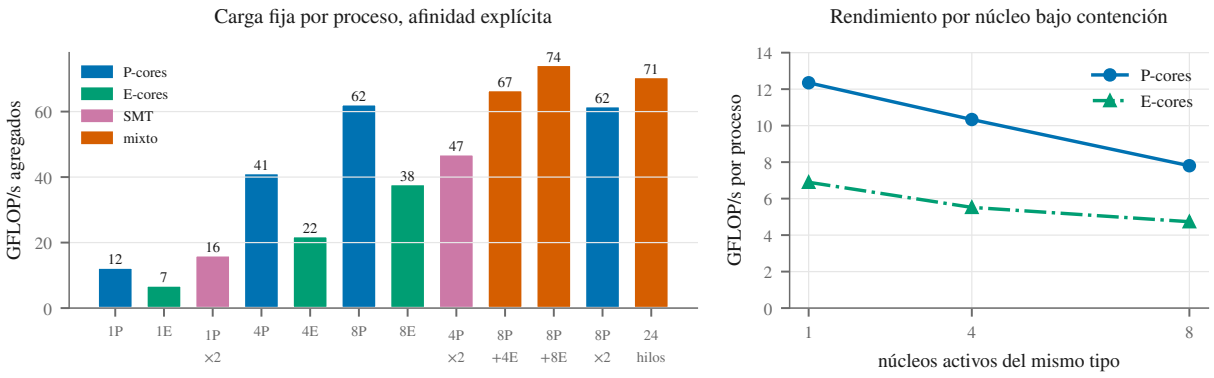


Figura 9: Rendimiento con la misma carga por proceso (1024 consultas $\times$ 160 000 puntos) y afinidad explícita a P-cores, E-cores o hilos SMT. Izquierda: GFLOP/s agregados. Derecha: GFLOP/s por proceso al aumentar los núcleos activos del mismo tipo.

El material del curso advierte sobre este fenómeno en la diapositiva “procesos vs. hilos” de la Unidad 4 [27, p. 22]:

> *"No se recomienda usar todos los hilos de un CPU por proceso porque más hilos por proceso no significa más rendimiento. Pasado cierto punto, cada hilo extra aporta poco o incluso resta. Debido a: Ancho de banda de memoria (... tiene muchísimos más hilos que canales para alimentarlos), NUMA, SMT (Simultaneous Multithreading) no equivale a núcleos reales, Sobrecarga de OpenMP/MPI y Desbalance de carga. El óptimo suele estar en un punto intermedio..."*

Para verificar experimentalmente estos factores se fijó cada proceso a una CPU lógica concreta con la misma carga de trabajo. Los resultados confirman tres de ellos (el i7-14650HX tiene un único nodo de memoria, por lo que NUMA no aplica, y la sobrecarga de MPI ya se midió como menor al 1 % en la Sec. 3.8):

1. **Saturación del ancho de banda de memoria.** Un P-core aislado rinde 12.3 GFLOP/s, pero baja a 10.3 con 4 P-cores activos y a 7.8 con 8; los E-cores pasan de 6.9 a 4.7. El rendimiento de cada núcleo disminuye cuando otros usan la memoria a la vez, aunque cada uno tenga su propio núcleo físico.

2. **SMT no equivale a núcleos reales.** Dos procesos en los dos hilos de un mismo P-core suman 16.1 GFLOP/s, apenas un 30 % más que uno solo. Ocho procesos en 4 P-cores con SMT rinden 47.0, frente a 62.2 en 8 P-cores físicos. Pasar de 16 a 24 procesos (agregando los hilos SMT) no mejora el rendimiento (74.3 → 70.6).

3. **Desbalance de cargas (P-cores frente a E-cores).** Un E-core rinde la mitad que un P-core (6.9 frente a 12.3 GFLOP/s). Con reparto por bloques iguales, en la configuración 8P+8E los procesos en E-cores rinden 3.2 GFLOP/s y los de P-cores 6.0: el tiempo total lo fija el más lento y los rápidos esperan en `Gatherv`. Esto explica el crecimiento de T_{comm} medido para $s \geq 12$ (Tabla 4), que es principalmente espera y no transferencia.

El óptimo medido, $s^* = 12$, es exactamente el "punto intermedio" que anticipa la diapositiva: ni los 8 P-cores ni los 24 hilos. Las tres regiones de la Fig. 2 quedan así explicadas: hasta $s = 8$ escalan los P-cores, limitados por el ancho de banda compartido; de 8 a 16 los E-cores aportan poco y desbalancean la carga; por encima de 16 los hilos SMT comparten unidades de ejecución y no añaden rendimiento.

3.10. Comparación de descomposiciones: Beta 3 frente a variante DAG

Tabla 9: Beta 3 (replica X_{tr}) frente a variante DAG (reparte X_{tr}), $N = 200\,000$.

	T_{total} [s]		T_{comm} [s]		
s	Beta 3	DAG	Beta 3	DAG	Fusión DAG [s]
1	98.46	96.52	0.003	0.016	0.006
8	19.97	19.03	0.749	0.161	0.018
12	18.60	18.01	0.972	0.626	0.025
16	18.90	17.82	1.195	0.387	0.043
24	19.26	18.23	2.705	3.187	0.056

La variante DAG es entre un 2 y un 6 % más rápida y alcanza más GFLOP/s (70 frente a 66). Dos factores lo explican: (i) evita el `Bcast` de X_{tr}, que bajo el modelo lineal cuesta $(s-1)n_{\text{train}}d\beta$; y (ii) cada proceso recorre sólo n_{train}/s puntos de entrenamiento, por lo que su bloque de datos es s veces menor y reduce el tráfico hacia memoria, el recurso que limita la escalabilidad. Su fusión serial crece linealmente con s, como predice la Ec. (4), pero con $k = 3$ es despreciable (0.056 s con $s = 24$). Ambas versiones producen predicciones idénticas.

3.11. Límites de escalabilidad y mejoras propuestas

- **Límite principal:** el ancho de banda de memoria compartido. Aumentar los procesos por encima de $s^* \approx 12$ no reduce el tiempo en esta máquina.
- **Balance dinámico o ponderado:** asignar a cada proceso un número de consultas proporcional a su rendimiento (P-core $\approx 2\times$ E-core) en `Scatterv`, reduciendo la espera en `Gatherv`.
- **Afinidad explícita:** con $s \leq 8$, fijar los procesos a P-cores físicos evita que el sistema operativo los asigne a hilos SMT o E-cores.
- **Bloqueo por caché:** recorrer X_{tr} en bloques que quepan en la caché L2 y reutilizarlos para varias consultas aumenta la intensidad aritmética y alivia el bus.
- **Memoria distribuida:** en un clúster, cada nodo aporta su propio canal de memoria y la variante DAG eliminaría la réplica de X_{tr}; allí el modelo en árbol sería el adecuado y la escalabilidad débil mejoraría según la Ley de Gustafson.

4. Conclusiones

1. Se paralelizó KNN con MPI siguiendo PCAM y un diseño PRAM CREW, expresado con el formalismo `forall` ... `pardo` en los Algoritmos 4–6. Las tres versiones beta y la variante DAG producen las mismas predicciones que el código secuencial y que scikit-learn.
2. El desarrollo en versiones mostró que la vectorización (Beta 2) reduce el tiempo $\sim$156 veces y la comunicación con buffers (Beta 3) un 33 % adicional; el speedup absoluto frente al código original llega a $\sim$700 (Beta 3) y $\sim$1000 (DAG) con 8 procesos.
3. La normalización $T_p(s) = k_1 W/s + k_2 T_{\text{comm}}(s)$ con el modelo de comunicación lineal reproduce los datos con un 6 % de error y predice analíticamente $s^* \approx 11.6$, en acuerdo con el mínimo experimental $s = 12$, que es la cantidad óptima de procesos estimada.
4. El speedup se satura en $\sim$5.3 ($f_s \approx 0.14$ según Amdahl), con eficiencia 0.21 para $s = 24$; la escalabilidad débil tampoco se sostiene (Gustafson, $E = 0.20$ con $s = 24$). La causa, demostrada experimentalmente, no es la comunicación MPI sino el hardware compartido: saturación del ancho de banda de memoria, SMT que no equivale a núcleos reales y desbalance entre P-cores y E-cores.
5. En memoria compartida las colectivas siguen el modelo lineal $O(s)$ ($R^2 = 0.994$) y no el de árbol binario: las copias compiten por el bus DDR5, una manifestación directa del cuello de botella de von Neumann.

5. Bibliografía y uso de fuentes externas

Esta sección detalla qué fuente se empleó, con qué propósito y su impacto en el trabajo.

5.1. Material del curso

El material de clase de Fiestas [3, 21, 24, 10, 28, 27] define el marco teórico y la notación de todo el informe:

- Unidad 1 [3]: DAG, taxonomía de Flynn, metodología de Foster (Sec. 2.2) y el cuello de botella de von Neumann usado para interpretar k_2 y el modelo lineal.

- Unidad 2.1 [24]: speedup, eficiencia, FLOPs, Ley de Amdahl (escalabilidad fuerte, Ec. 11) y Ley de Gustafson (escalabilidad débil, Ec. 12).
- Unidad 2 [21]: diapositivas "Comunicación lineal" y "Comunicación en árbol (binario)" (págs. 7–8), base de las Ecs. 7–8, de la normalización de la Sec. 3.3 y del contraste experimental de la Sec. 3.8.
- Unidad 2.2–2.3 [10]: PRAM CREW, formalismo `pardo` y teorema de Brent.
- Unidad 3 [28]: colectivas MPI (`Bcast`, `Scatter(v)`, `Gather(v)`) usadas en los Algoritmos 5–6.
- Unidad 4 [27]: diapositiva "procesos vs. hilos" (pág. 22), citada textualmente en la Sec. 3.9; sus factores (ancho de banda de memoria, SMT, desbalance de carga) se verificaron experimentalmente con el estudio de afinidad.
- Enunciado del proyecto [4]: código secuencial, DAG de referencia y preguntas de análisis.

5.2. Literatura científica y documentación técnica

- Fix y Hodges [1], Cover y Hart [2]: fundamento de KNN y de su naturaleza no paramétrica, que concentra el cómputo en la consulta.
- JáJá [9] y Brent [11]: PRAM CREW, sintaxis `pardo` y cota del tiempo paralelo (Ec. 2).
- Grama et al. [12] y Thakur, Rabenseifner y Gropp [26]: costo de las colectivas en árbol, contrastado con nuestras mediciones (Sec. 3.8).
- Valiant [29] y Culler et al. [30]: motivación de modelos con costo explícito de comunicación (BSP, LogP).
- Foster [7], Flynn [8], Amdahl [23] y Gustafson [25]: metodología PCAM, clasificación MIMD/SPMD y leyes de escalabilidad.
- Backus [22]: formulación del cuello de botella de von Neumann.
- Pacheco [31] y Quinn [32]: referencia de programación MPI.
- Aparicío, Blanquer y Hernández [33]: validan la estrategia de replicar X_{tr} y repartir las consultas; Patwary et al. [34] y Xiao y Biros [35]: cuellos de botella de KNN distribuido a gran escala.
- Documentación técnica: mpi4py [17, 16] (semántica de `Bcast`, `Scatterv`, `Gatherv` con buffers), Intel MPI [18] (colectivas intranodo), Intel [13] (arquitectura del i7-14650HX), Microsoft [14] (topología P/E), scikit-learn [5] (validación), NumPy [15], SciPy [19] (ajustes por mínimos cuadrados) y Matplotlib [20] (figuras).

5.3. Uso de herramientas de asistencia y modelos de lenguaje

En cumplimiento de las pautas del curso, se declara el uso del modelo de lenguaje Claude [36], a través del entorno Claude Code, como herramienta de apoyo.

Referencias

[1] Evelyn Fix y Joseph L. Hodges. *Discriminatory Analysis: Nonparametric Discrimination, Consistency Properties*. Inf. téc. 4. Randolph Field, TX: USAF School of Aviation Medicine, 1951.

[2] Thomas M. Cover y Peter E. Hart. «Nearest Neighbor Pattern Classification». En: *IEEE Transactions on Information Theory* 13.1 (1967), págs. 21-27. DOI: 10.1109/TIT.1967.1053964.

[3] José Fiestas. *Unidad 1: Fundamentos de paralelismo – DAG, taxonomía de Flynn y método de Foster*. Material de clase, CS4052, Universidad de Ingeniería y Tecnología (UTEC). Lima, Perú, 2026.

[4] José Fiestas. *Proyecto – Laboratorio: CS4052 Computación Paralela y Distribuida (2026-II)*. Enunciado del curso, Universidad de Ingeniería y Tecnología (UTEC). Lima, Perú, 2026.

[5] Fabian Pedregosa y col. «Scikit-learn: Machine Learning in Python». En: *Journal of Machine Learning Research* 12 (2011), págs. 2825-2830.

[6] Ethem Alpaydin y Cenk Kaynak. *Optical Recognition of Handwritten Digits*. UCI Machine Learning Repository. Conjunto de datos. 1998. DOI: 10.24432/C50P49.

[7] Ian Foster. *Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering*. Reading, MA: Addison-Wesley, 1995. ISBN: 978-0201575941.

[8] Michael J. Flynn. «Some Computer Organizations and Their Effectiveness». En: *IEEE Transactions on Computers* C-21.9 (1972), págs. 948-960. DOI: 10.1109/TC.1972.5009071.

[9] Joseph JáJá. *An Introduction to Parallel Algorithms*. Reading, MA: Addison-Wesley, 1992. ISBN: 978-0201548563.

[10] José Fiestas. *Unidad 2.2–2.3: Modelos computacionales en paralelo (PRAM, BSP, LogP)*. Material de clase, CS4052, Universidad de Ingeniería y Tecnología (UTEC). Lima, Perú, 2026.

[11] Richard P. Brent. «The Parallel Evaluation of General Arithmetic Expressions». En: *Journal of the ACM* 21.2 (1974), págs. 201-206. DOI: 10.1145/321812.321815.

[12] Ananth Grama y col. *Introduction to Parallel Computing*. 2.ª ed. Addison-Wesley, 2003.

[13] Intel Corporation. *Intel Core i7-14650HX Processor – Product Specifications*. 2024. URL: https://www.intel.com/content/www/us/en/products/sku/235997/intel-core-i714650hx-processor-30m-cache-up-to-5-20-ghz/specifications.html (visitado 08-11-2026).

[14] Microsoft. *GetSystemCpuSetInformation function (processthreadsapi.h)*. 2024. URL: https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-getsystemcpusetinformation (visitado 08-11-2026).

[15] Charles R. Harris y col. «Array Programming with NumPy». En: *Nature* 585.7825 (2020), págs. 357-362. DOI: 10.1038/s41586-020-2649-2.

[16] Lisandro Dalcín y Yao-Lung L. Fang. «mpi4py: Status Update After 12 Years of Development». En: *Computing in Science & Engineering* 23.4 (2021), págs. 47-54. DOI: 10.1109/MCSE.2021.3083216.

[17] Lisandro Dalcín. *MPI for Python (mpi4py) documentation*. 2025. URL: https://mpi4py.readthedocs.io/en/stable/ (visitado 08-11-2026).

[18] Intel Corporation. *Intel MPI Library Developer Reference for Windows OS*. 2025. URL: https://www.intel.com/content/www/us/en/docs/mpi-library/developer-reference-windows/ (visitado 08-11-2026).

[19] Pauli Virtanen y col. «SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python». En: *Nature Methods* 17 (2020), págs. 261-272. DOI: 10.1038/s41592-019-0686-2.

[20] John D. Hunter. «Matplotlib: A 2D Graphics Environment». En: *Computing in Science & Engineering* 9.3 (2007), págs. 90-95. DOI: 10.1109/MCSE.2007.55.

[21] José Fiestas. *Unidad 2: Diseño en paralelo – Métodos de paralelismo, modelos PRAM y comunicación lineal vs. árbol*. Material de clase, CS4052, Universidad de Ingeniería y Tecnología (UTEC). Lima, Perú, 2026.

[22] John Backus. «Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs». En: *Communications of the ACM* 21.8 (1978), págs. 613-641. DOI: 10.1145/359576.359579.

[23] Gene M. Amdahl. «Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities». En: *Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (AFIPS '67)*. Nueva York, NY: ACM, 1967, págs. 483-485. DOI: 10.1145/1465482.1465560.

[24] José Fiestas. *Unidad 2.1: Diseño en paralelo – velocidad, eficiencia, escalabilidad y FLOPs*. Material de clase, CS4052, Universidad de Ingeniería y Tecnología (UTEC). Lima, Perú, 2026.

[25] John L. Gustafson. «Reevaluating Amdahl's Law». En: *Communications of the ACM* 31.5 (1988), págs. 532-533. DOI: 10.1145/42411.42415.

[26] Rajeev Thakur, Rolf Rabenseifner y William Gropp. «Optimization of Collective Communication Operations in MPICH». En: *The International Journal of High Performance Computing Applications* 19.1 (2005), págs. 49-66. DOI: 10.1177/1094342005051521.

[27] José Fiestas. *Unidad 4: Comunicación y coordinación – Arquitecturas paralelas híbridas, procesos vs. hilos y memoria compartida*. Material de clase, CS4052, Universidad de Ingeniería y Tecnología (UTEC). Lima, Perú, 2026.

[28] José Fiestas. *Unidad 3: Paralelismo distribuido – MPI, comunicación colectiva y tipos derivados*. Material de clase, CS4052, Universidad de Ingeniería y Tecnología (UTEC). Lima, Perú, 2026.

[29] Leslie G. Valiant. «A Bridging Model for Parallel Computation». En: *Communications of the ACM* 33.8 (1990), págs. 103-111. DOI: 10.1145/79173.79181.

[30] David Culler y col. «LogP: Towards a Realistic Model of Parallel Computation». En: *Proceedings of the Fourth ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPOPP '93)*. Nueva York, NY: ACM, 1993, págs. 1-12. DOI: 10.1145/155332.155333.

[31] Peter S. Pacheco. *An Introduction to Parallel Programming*. Burlington, MA: Morgan Kaufmann, 2011. ISBN: 978-0-12-374260-5.

[32] Michael J. Quinn. *Parallel Programming in C with MPI and OpenMP*. Nueva York, NY: McGraw-Hill, 2003. ISBN: 978-0071232654.

[33] Gabriel Aparicío, Ignacio Blanquer y Vicente Hernández. «A Parallel Implementation of the K Nearest Neighbours Classifier in Three Levels: Threads, MPI Processes and the Grid». En: *High Performance Computing for Computational Science – VECPAR 2006*. Vol. 4395. Lecture Notes in Computer Science. Berlín: Springer, 2007, págs. 225-235. DOI: 10.1007/978-3-540-71351-7_18.

[34] Md. Mostofa Ali Patwary y col. «PANDA: Extreme Scale Parallel K-Nearest Neighbor on Distributed Architectures». En: *2016 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2016, págs. 494-503. DOI: 10.1109/IPDPS.2016.57.

[35] Bo Xiao y George Biros. «Parallel Algorithms for Nearest Neighbor Search Problems in High Dimensions». En: *SIAM Journal on Scientific Computing* 38.5 (2016), S667-S699. DOI: 10.1137/15M1026377.

[36] Anthropic. *Claude (versión Opus 5.5) mediante Claude Code*. Modelo de lenguaje de gran escala y entorno agéntico de programación. 2026. URL: https://claude.ai (visitado 08-11-2026).